UNIVERSIDAD NACIONAL DE COSTA RICA
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
ESCUELA DE INFORMÁTICA
ARQUITECTURA DE COMPUTADORAS (EIF205)

INFORME TÉCNICO

Proyecto 2: Maze8086

Juego de laberinto en lenguaje ensamblador 8086

Integrantes del grupo:

Belén Sandoval Méndez

Isaac Altamirano Gómez

Juan José Fernández Campos

Profesor:

César Valverde Canossa

Fecha de entrega: 8 de noviembre de 2026

1. Introducción

Maze8086 es un juego de laberinto en dos dimensiones escrito en lenguaje ensamblador Intel 8086. El jugador recorre un mapa de 20 columnas por 15 filas, recoge cinco monedas y busca la salida mientras evita trampas que le quitan vidas. El programa funciona en modo texto de 80 × 25 caracteres, recibe las órdenes desde el teclado y muestra durante la partida los puntos, el récord, las vidas, los movimientos, las monedas pendientes y el tiempo transcurrido. Los eventos principales tienen efectos de sonido generados con el parlante del PC.

El propósito del proyecto es aplicar los temas del curso en un programa interactivo: direccionamiento y alineación de memoria, uso de registros, procedimientos, macros, manejo de la pila, interrupciones del BIOS y de DOS, puertos de entrada y salida, y medición del tiempo. El código se dividió en un archivo principal y cuatro módulos que se integraron en una sola versión.

Este informe describe la arquitectura del programa, la distribución de los datos en memoria, las rutinas de cada módulo y las pruebas realizadas. Cada prueba se acompaña de una captura numerada de E01 a E28.

1.1. Reglas del juego

- Cada moneda (C) suma 10 puntos y desaparece del mapa al recogerla.
- La salida (S) solo se abre cuando no quedan monedas y suma 50 puntos. Si se intenta entrar antes, el jugador no se mueve y suena un tono grave.
- Al ganar se suma un bono de tiempo igual a 120 menos los segundos usados. Si la partida dura 120 segundos o más, el bono es 0. Una victoria vale entre 100 y 220 puntos.
- El cronómetro solo avanza durante la partida; no cuenta el tiempo en el menú ni en la pausa.
- Las paredes (#) bloquean el paso, no quitan vidas y no cuentan como movimiento.
- Cada trampa (X) quita una de las tres vidas, devuelve al jugador al inicio y desaparece. Al perder la tercera vida la partida termina en derrota.
- El récord guarda el mayor puntaje obtenido mientras el programa sigue abierto.

1.2. Controles

Pantalla	Teclas
Menú	1 inicia una partida. ESC cierra el programa.
Partida	Flechas o W, A, S, D mueven al jugador. P pausa. Q o ESC regresan al menú.
Pausa	P o Enter continúan. Q o ESC regresan al menú.
Victoria o derrota	R reinicia la partida. M o Q regresan al menú. ESC cierra el programa.

1.3. Entorno de desarrollo

El programa se ensambla como archivo COM con EMU8086 4.08. El código fuente está en maze.asm, interfaz.inc, movimiento.inc, juego.inc y sonido.inc. El ejecutable final, build/MAZE.COM, ocupa 3865 bytes. La versión final se probó en el emulador de EMU8086 y en DOSBox 0.74-3; las pruebas de la versión anterior se hicieron en DOSBox-X 2026.10.01.

2. Arquitectura del software

2.1. Organización en módulos

El archivo principal contiene los datos compartidos, el mapa y el ciclo de estados. Los otros cuatro archivos se incluyen con INCLUDE y cada uno define sus rutinas dentro de una macro (DEFINE_INTERFACE, DEFINE_MOVIMIENTO, DEFINE_JUEGO y DEFINE_SONIDO). EMU8086 no reconocía algunas rutinas incluidas directamente, por lo que cada macro se invoca una vez desde maze.asm.

Archivo	Responsabilidad	Integrante
maze.asm	Datos compartidos, mapa, arranque y ciclo de estados	Integración del equipo
interfaz.inc	Menú, tablero, marcador con reloj, pausa y pantallas finales	Belén Sandoval
movimiento.inc	Lectura de teclas, límites y colisiones	Juan José Fernández
juego.inc	Vidas, monedas, puntos, récord, victoria, reinicio y cronómetro	Isaac Altamirano
sonido.inc	Macro SONAR y efectos con el parlante del PC	Isaac Altamirano

El mapa y la presentación están separados. Las reglas consultan únicamente el arreglo mapa; la interfaz lo recorre para dibujarlo, pero no lo modifica. Cambiar el aspecto de una celda no altera su comportamiento en las colisiones.

2.2. Arranque y ciclo de juego

DOS carga el programa COM a partir del desplazamiento 0100h. La primera instrucción salta a la etiqueta inicio para no ejecutar los datos ni las rutinas que están antes. En inicio se copia CS a DS, se limpia la bandera de dirección y se llama a ui_init, que selecciona el modo texto 03h y oculta el cursor.

Durante la partida, el ciclo esperar_jugada consulta el teclado sin detenerse, con INT 16h función 01h. Mientras no hay tecla, llama a actualizar_tiempo, que lee el contador del BIOS y redibuja el reloj cuando cambia el segundo. Cuando hay una tecla, la retira con INT 16h función 00h y la entrega a procesar_movimiento. Esta rutina calcula la posición candidata, la valida y aplica las reglas. Al terminar, el ciclo principal consulta la variable estado y decide si redibuja el tablero o muestra la pausa, la victoria, la derrota o el menú.

El menú, la pausa y las pantallas finales no necesitan reloj, así que esperan la tecla con INT 16h función 00h, que detiene el programa hasta que se pulse una.

2.3. Estados de la aplicación

Valor	Estado	Transiciones
0	Menú	1 inicia una partida. ESC cierra el programa.
1	Partida	P abre la pausa. Q o ESC regresan al menú. Las reglas pueden cambiar a victoria o derrota.
2	Victoria	R reinicia. M o Q regresan al menú. ESC cierra el programa.
3	Derrota	R reinicia. M o Q regresan al menú. ESC cierra el programa.

Valor	Estado	Transiciones
4	Pausa	P o Enter reanudan la partida. Q o ESC regresan al menú.

Las rutinas de reglas solo actúan cuando estado vale 1. Esto impide sumar puntos o perder vidas después de que la partida terminó.

2.4. Compilación y ejecución

- Colocar maze.asm, interfaz.inc, movimiento.inc, juego.inc y sonido.inc en la misma carpeta.
- Abrir maze.asm en EMU8086 4.08, compilar como COM y guardar el resultado como build/MAZE.COM.
- Con DOSBox-X, ejecutar JUGAR.bat. Este archivo abre DOSBox-X con build/play.conf, que monta la carpeta build y ejecuta MAZE.COM.
- Con DOSBox, arrastrar build/MAZE.COM sobre el ícono del programa, o montar la carpeta build con mount c y ejecutar MAZE.COM.
- En el emulador de EMU8086, la ventana emulator screen debe estar en 80 × 25 caracteres (sección 5.4).

3. Mapeo de memoria

3.1. Segmentos

Un programa COM usa un solo segmento de 64 KB para código, datos y pila. DS apunta a los datos del juego. ES se usa de forma temporal para copiar el mapa y para escribir en la memoria de video. La pila la prepara DOS al cargar el programa (SS:SP). En modo real, la dirección física se calcula como segmento × 16 + desplazamiento.

3.2. Variables del juego

Los desplazamientos de la tabla se tomaron del archivo de símbolos que genera EMU8086 (MAZE.COM.symbol, figura E21).

Variable	Tamaño	Offset	Uso
relleno_alineacion	1 byte	0103h	Relleno para alinear las variables de 16 bits
puntos, record_max, movimientos	2 bytes c/u	0104h, 0106h, 0108h	Puntaje, récord de la sesión y movimientos válidos
segundos	2 bytes	010Ah	Tiempo de la partida mostrado en pantalla
ticks_partida, tick_anterior	2 bytes c/u	010Ch, 010Eh	Ticks acumulados en la partida y última lectura del BIOS
jugador_x, jugador_y	1 byte c/u	0110h, 0111h	Columna y fila del jugador, desde cero
vidas, monedas	1 byte c/u	0112h, 0113h	Vidas restantes y monedas pendientes
estado	1 byte	0114h	Estado actual de la aplicación

Variable	Tamaño	Offset	Uso
mapa_original	300 bytes	0115h	Diseño fijo del laberinto
mapa	300 bytes	0241h	Copia de trabajo donde se consumen monedas y trampas
ui_number_buffer	6 bytes	072Ah	Cinco dígitos ASCII y el terminador

3.3. Alineación de memoria

El 8086 tiene un bus de datos de 16 bits. Una variable word que empieza en una dirección par se lee o escribe en un solo acceso al bus; si empieza en una dirección impar, el procesador necesita dos accesos. En la versión anterior, el salto inicial ocupaba 3 bytes y le seguían cuatro variables de un byte, por lo que puntos, record_max y movimientos quedaban en 0107h, 0109h y 010Bh, todas impares.

La versión final agrega el byte relleno_alineacion después del salto y declara primero todas las variables word. Así quedan en direcciones pares, de 0104h a 010Eh, y las variables de un byte van después, donde la alineación no importa. Se usó un byte explícito en lugar de una directiva de alineación para que el resultado no dependa del ensamblador; la figura E21 confirma las direcciones generadas por EMU8086.

3.4. Mapa y memoria de video

El mapa se guarda como 15 filas consecutivas de 20 bytes. La celda de la columna x y la fila y está en el índice y × 20 + x. Por ejemplo, la primera moneda, en la posición (4, 1), está en el índice 24. El jugador no se guarda dentro del mapa: sus coordenadas están en jugador_x y jugador_y, y la interfaz lo dibuja encima.

La pantalla de 80 × 25 caracteres ocupa 4000 bytes a partir del segmento B800h. Cada posición usa dos bytes: el carácter y su color. La rutina ui_offset calcula la dirección como fila × 160 + columna × 2. El tablero empieza en la fila 6 y la columna 5, y cada celda del mapa se dibuja con dos caracteres para que el laberinto no se vea angosto.

4. Descripción de módulos y rutinas

4.1. maze.asm

Contiene los datos, el mapa y el ciclo principal. La etiqueta mostrar_menu dibuja el menú y espera la opción. nueva_partida llama a reiniciar_juego; dibujar_partida y esperar_jugada forman el ciclo de la partida. mostrar_pausa y esperar_final atienden la pausa y las pantallas de victoria o derrota. Al salir, ui_shutdown restaura el modo de video y INT 21h con AX = 4C00h devuelve el control a DOS.

4.2. interfaz.inc

Rutina	Función
ui_init / ui_shutdown	Seleccionan el modo texto 03h con INT 10h; ui_init además oculta el cursor.
ui_offset	Convierte fila y columna (DH, DL) en un desplazamiento de video en DI.
ui_fill	Escribe un carácter con color CX veces usando REP STOSW.

Rutina	Función
ui_text	Escribe una cadena terminada en cero desde DS:SI.
ui_number	Convierte un número de 16 bits a cinco dígitos dividiendo entre 10.
ui_clear, ui_box	Limpian la pantalla y dibujan marcos de doble línea.
ui_menu	Dibuja el menú principal con el título y las instrucciones.
ui_board, ui_hud	Dibujan el laberinto, el jugador y el marcador lateral.
ui_tiempo	Redibuja solo los segundos del marcador, sin volver a dibujar el tablero.
ui_pause, ui_win, ui_lose	Llaman a ui_end_screen con el tipo de pantalla que corresponde.

4.3. movimiento.inc

procesar_movimiento lee el código de exploración en AH para las flechas y el carácter en AL para las letras. Las letras se convierten a minúscula con OR AL, 20h, por lo que W y w producen el mismo efecto. La rutina calcula la posición candidata en BL y BH y la compara con ANCHO y ALTO mediante saltos sin signo. Una coordenada negativa se convierte en 255 y también queda rechazada. Luego calcula el índice y × 20 + x y revisa la celda:

- Si es pared, cancela el movimiento sin cambiar coordenadas ni contador.
- Si es la salida y quedan monedas, cancela el movimiento y llama a sonido_bloqueo. Si no quedan, mueve al jugador y llama a ganar_partida.
- En otro caso actualiza las coordenadas, llama a contar_movimiento y, si la celda tenía moneda o trampa, llama a sumar_moneda o perder_vida y limpia la celda.

4.4. juego.inc

Rutina	Función
reiniciar_juego	Copia los 300 bytes de mapa_original a mapa con REP MOVSB y restablece posición, vidas, monedas, puntos, movimientos y tiempo. No modifica record_max.
sumar_moneda	Resta una moneda pendiente, suma 10 puntos, actualiza el récord y toca el sonido de moneda.
perder_vida	Resta una vida. Si quedan vidas devuelve al jugador a (1, 1) con el sonido de trampa; si no, cambia el estado a derrota con su sonido.
ganar_partida	Toma el tiempo final, suma 50 puntos y el bono de tiempo, actualiza el récord, cambia el estado a victoria y toca la melodía de victoria.
actualizar_record	Copia puntos en record_max cuando el puntaje es mayor.
iniciar_reloj	Guarda la lectura actual del BIOS como punto de partida del cronómetro.
actualizar_tiempo	Suma los ticks transcurridos desde la última lectura y calcula los segundos.

4.5. Cronómetro

INT 1Ah con AH = 00h devuelve en CX:DX la cantidad de ticks desde la medianoche; el reloj del BIOS produce unos 18,2 ticks por segundo. El cronómetro usa solo la palabra baja (DX). actualizar_tiempo resta la lectura anterior a la actual y suma la diferencia a ticks_partida. La resta sin signo da el resultado correcto aunque DX haya pasado de FFFFh a 0000h entre dos lecturas.

Los segundos se calculan como ticks × 10 / 182, con MUL y DIV de 16 bits, para no usar números con decimales. Si una lectura muestra un salto mayor a 91 ticks (5 segundos), se descarta: eso solo ocurre cuando el contador del BIOS vuelve a cero a medianoche o cuando el emulador estuvo detenido.

El tiempo solo se acumula mientras estado vale 1. Al reanudar desde la pausa, iniciar_reloj toma una lectura nueva, de modo que los segundos en pausa no se suman. El bono se calcula en ganar_partida restando segundos a la constante TIEMPO_BONO (120); si la resta activa la bandera de acarreo o da cero, la instrucción JBE salta la suma del bono.

4.6. sonido.inc

El sonido usa el canal 2 del temporizador programable 8253/8254, conectado al parlante del PC. sonido_tono escribe 0B6h en el puerto de control 43h (canal 2, modo de onda cuadrada), envía el divisor por el puerto 42h en dos bytes y enciende el parlante poniendo en 1 los bits 0 y 1 del puerto 61h. La frecuencia resultante es 1 193 180 / divisor. Después espera la duración indicada con esperar_ticks y apaga el parlante borrando esos dos bits.

esperar_ticks mide la duración con INT 1Ah, igual que el cronómetro, en lugar de un ciclo de retraso con un contador fijo. Así la duración no depende de la velocidad del procesador ni de los ciclos configurados en el emulador.

Rutina	Cuándo se usa	Notas
sonido_moneda	Al recoger una moneda	Do6 y Mi6, cortas
sonido_trampa	Al perder una vida con vidas restantes	Mi4 y Do4
sonido_bloqueo	Al intentar salir con monedas pendientes	Tono grave de 131 Hz
sonido_victoria	Al ganar	Arpegio Do5, Mi5, Sol5, Do6
sonido_derrota	Al perder la última vida	Sol4, Mi4, Do4 descendente

4.7. Uso de la pila y macros

CALL guarda la dirección de retorno en la pila y RET la recupera. Cada rutina guarda con PUSH los registros que modifica y los restaura con POP en orden inverso antes de RET, de modo que quien la llama conserva sus valores. reiniciar_juego, ui_fill y ui_text también guardan ES, porque lo cambian para copiar el mapa o escribir en B800h; esperar_ticks guarda DI, que usa para conservar el tick inicial. Los parámetros se pasan en registros (por ejemplo, DH y DL para la posición y BL para el color).

La macro UI_LABEL reemplaza la secuencia repetida para imprimir un mensaje: cargar fila y columna en DH y DL, la dirección de la cadena en SI y el color en BL, y llamar a ui_text. Se usa más de cincuenta veces en la interfaz. La macro SONAR cumple la misma función para el sonido: recibe el divisor de la nota y su duración, los carga en BX y CX y llama a sonido_tono. Las macros DEFINE_INTERFACE, DEFINE_MOVIMIENTO, DEFINE_JUEGO y DEFINE_SONIDO agrupan el código de cada módulo para incluirlo en el ensamblado.

5. Bitácora de pruebas

Las pruebas se hicieron de forma manual, con el teclado, observando el marcador y las pantallas. En cada caso se anotó la acción, el resultado esperado y lo que mostró la pantalla.

Hubo dos rondas. La primera se hizo sobre la versión anterior (MAZE.COM de 3376 bytes, en DOSBox-X 2026.10.01) y cubre el movimiento, las colisiones, las monedas, las trampas y los estados (secciones 5.1 y 5.2, figuras E01 a E17). La versión final no cambió esas reglas. La segunda ronda se hizo sobre la versión final (MAZE.COM de 3865 bytes, en el emulador de EMU8086 y en DOSBox 0.74-3) y cubre la alineación, la compatibilidad con EMU8086, el cronómetro, el bono y el sonido (sección 5.3, figuras E20 a E28).

5.1. Casos de colisiones y límites

Fig.	Acción	Esperado	Observado
E04	Desde el inicio, pulsar W contra la pared.	El jugador no se mueve.	Misma posición que en E03 y 0 movimientos.
E05	Pulsar D tres veces desde el inicio.	Recoger la primera moneda y sumar 10.	10 puntos, 4 monedas pendientes y 3 movimientos.
E06	Salir de la celda de la moneda y volver con A y D.	No volver a sumar puntos.	10 puntos, 4 monedas pendientes y 5 movimientos.
E10	Desde el inicio, pulsar S, S, D, D.	Perder una vida y volver al inicio.	2 vidas, jugador en el inicio y la trampa ya no aparece.
E11	Repetir S, S, D, D.	Pasar por la celda sin perder otra vida.	El jugador llega a la celda y conserva 2 vidas.
E12	Intentar entrar a S con monedas pendientes.	Seguir en partida sin bonificación.	Jugador junto a la salida, 30 puntos y 2 monedas pendientes antes y después del intento.
E17	Intentar avanzar hacia el borde del laberinto.	El jugador permanece dentro del mapa.	El jugador queda dentro del laberinto, junto al borde.

5.2. Pruebas de estado

Fig.	Acción	Esperado	Observado
E01	Compilar la versión anterior en EMU8086.	Generar el archivo COM sin errores.	Ensamblado en cuatro pasadas; MAZE.COM de 3376 bytes.
E02	Ejecutar MAZE.COM.	Ver el menú principal.	Menú con las opciones 1 y ESC.
E03	Pulsar 1 en el menú.	Iniciar la partida con los valores base.	3 vidas, 5 monedas, 0 puntos y 0 movimientos.
E07, E08	Pulsar P, intentar mover y reanudar con Enter.	Pausar y continuar sin cambios.	Pantalla de pausa; al reanudar, misma celda de E06 con 10 puntos y 5 movimientos.
E09	Volver al menú e iniciar otra partida después de puntuar.	Reiniciar el puntaje y conservar el récord.	0 puntos, récord 10, 3 vidas y 5 monedas.
E13	Recoger las cinco monedas y entrar en S.	Ganar con 100 puntos.	Pantalla de victoria con 100 puntos y récord 100.
E14	Pulsar R en la pantalla de victoria.	Restaurar la partida y conservar el récord.	3 vidas, 5 monedas, 0 puntos, 0 movimientos y récord 100.

Fig.	Acción	Esperado	Observado
E15	Activar las tres trampas.	Pasar al estado de derrota.	Pantalla de Game Over con récord 100.
E16	Pulsar R en la pantalla de derrota.	Reiniciar sin cerrar el emulador.	Tablero inicial, 3 vidas, 5 monedas y récord 100.

5.3. Pruebas de la versión final

Fig.	Acción	Esperado	Observado
E20	Compilar maze.asm con los cuatro INC en EMU8086.	Generar el archivo COM sin errores.	Ensamblado en cuatro pasadas; MAZE.COM de 3865 bytes.
E21	Abrir MAZE.COM.symbol.	Variables word en direcciones pares.	puntos 0104h, record_max 0106h, movimientos 0108h, segundos 010Ah, ticks_partida 010Ch, tick_anterior 010Eh.
E22	Ejecutar el programa en el emulador de EMU8086.	Ver el menú completo.	Menú con las opciones y la línea del bono de tiempo.
E23	Pulsar 1 en el emulador de EMU8086, con la ventana en 80 × 25.	Ver el tablero y el marcador.	Tablero completo con la línea TIEMPO (SEG) en 0.
E24	En DOSBox, pulsar 1 en el menú.	Iniciar con el reloj en cero.	3 vidas, 5 monedas, 0 puntos, 0 movimientos y tiempo 0.
E25	Jugar unos segundos y recoger tres monedas.	El reloj avanza durante la partida.	Tiempo 15, 30 puntos, 2 monedas pendientes y 40 movimientos.
E26	Pulsar P y esperar en la pausa.	Mostrar la pausa con el tiempo detenido.	Pantalla de pausa con 30 puntos y tiempo 17.
E27	Pulsar Enter para continuar.	Volver a la partida sin sumar el tiempo en pausa.	Misma posición y marcador; tiempo 18, un segundo más que al pausar.
E28	Recoger las cinco monedas y entrar en S.	Ganar con 150 puntos más el bono (120 − segundos).	Victoria en 29 segundos con 191 puntos (100 + 91) y récord 191.
—	Recoger una moneda, tocar una trampa, intentar salir con monedas pendientes, ganar y perder.	Escuchar un sonido distinto en cada evento.	Se escuchan los cinco sonidos en DOSBox. El parlante no se puede mostrar en una captura.

El ejemplo de la figura E28 comprueba el cálculo del bono: la victoria fija 100 puntos (50 por monedas y 50 por la salida) y el bono es 120 − 29 = 91, para un total de 191.

5.4. Registro de errores encontrados y soluciones

Error	Impacto	Solución aplicada
Nombres de variables distintos entre módulos	La interfaz y las reglas no compartían posición, vidas, puntos ni récord.	Unificar los nombres de las variables compartidas en maze.asm.
Símbolos de celda inconsistentes	El mapa dibujado no coincidía con lo que revisaban las colisiones.	Usar los mismos símbolos en todos los módulos: # pared, C moneda, X trampa y S salida.

Error	Impacto	Solución aplicada
Ciclo de juego incompleto	El programa terminaba antes de procesar una partida.	Conectar el menú, la espera de teclado y las transiciones de estado.
Rutinas incluidas no reconocidas por EMU8086	El ensamblador no encontraba algunos procedimientos de los INC.	Agrupar cada módulo en una macro DEFINE_ y guardar los INC con saltos de línea CRLF.
Ruta con tildes al montar la carpeta	DOSBox-X no encontraba la carpeta del juego.	Montar build con una ruta relativa desde play.conf y abrirlo con JUGAR.bat.
Desalineación de memoria: puntos, record_max y movimientos en 0107h, 0109h y 010Bh	Cada lectura o escritura de esas variables necesitaba dos accesos al bus en un 8086.	Agregar un byte de relleno después del salto inicial y declarar primero las variables word (sección 3.3).
Tablero desplazado en el emulador de EMU8086	Con la ventana emulator screen en 86 × 30 o 110 × 36 caracteres, cada fila del tablero aparecía corrida.	El juego escribe filas de 80 caracteres en B800h. Se ajusta la ventana a 80 × 25 antes de ejecutar (figura E23).
Reloj sin actualizarse mientras se espera una tecla	Con INT 16h función 00h el programa se detiene hasta recibir una tecla, y el tiempo en pantalla no cambiaba.	Consultar el teclado con INT 16h función 01h y actualizar el reloj mientras no hay tecla.

5.5. Evidencias de la versión final

E20. Compilación de la versión final en EMU8086.

El ensamblador genera MAZE.COM de 3865 bytes y guarda el listado y la tabla de símbolos.

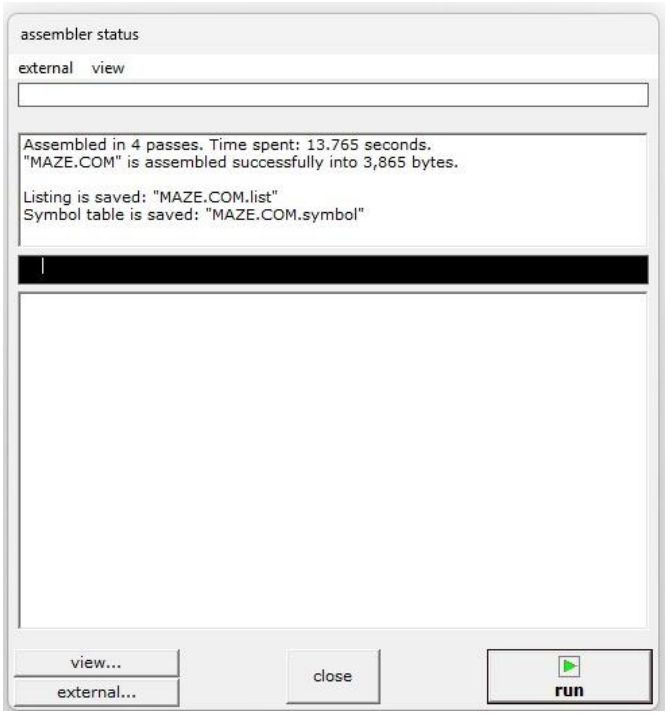


Figura E20. Compilación de la versión final en EMU8086.

E21. Tabla de símbolos con las variables alineadas.

Después del byte de relleno en 0103h, las seis variables de 16 bits ocupan direcciones pares de 0104h a 010Eh.

```
< THE SYMBOL TABLE >  MAZE.COM  --  emu8086 assembler version: 4.08
================================================================================
Name                 Offset   Size   Type   Segment
================================================================================
RELLENO_ALINEACION   00103    1      VAR    (NOSEG)
PUNTOS               00104    2      VAR    (NOSEG)
RECORD_MAX           00106    2      VAR    (NOSEG)
MOVIMIENTOS          00108    2      VAR    (NOSEG)
SEGUNDOS             0010A    2      VAR    (NOSEG)
TICKS_PARTIDA        0010C    2      VAR    (NOSEG)
TICK_ANTERIOR        0010E    2      VAR    (NOSEG)
JUGADOR_X            00110    1      VAR    (NOSEG)
JUGADOR_Y            00111    1      VAR    (NOSEG)
VIDAS                00112    1      VAR    (NOSEG)
MONEDAS              00113    1      VAR    (NOSEG)
ESTADO               00114    1      VAR    (NOSEG)
MAPA_ORIGINAL        00115    1      VAR    (NOSEG)
MAPA                 00241    1      VAR    (NOSEG)
```

Figura E21. Tabla de símbolos con las variables alineadas.

E22. Menú en el emulador de EMU8086.

El programa corre en el emulador de EMU8086 y muestra el menú con la línea del bono de tiempo.

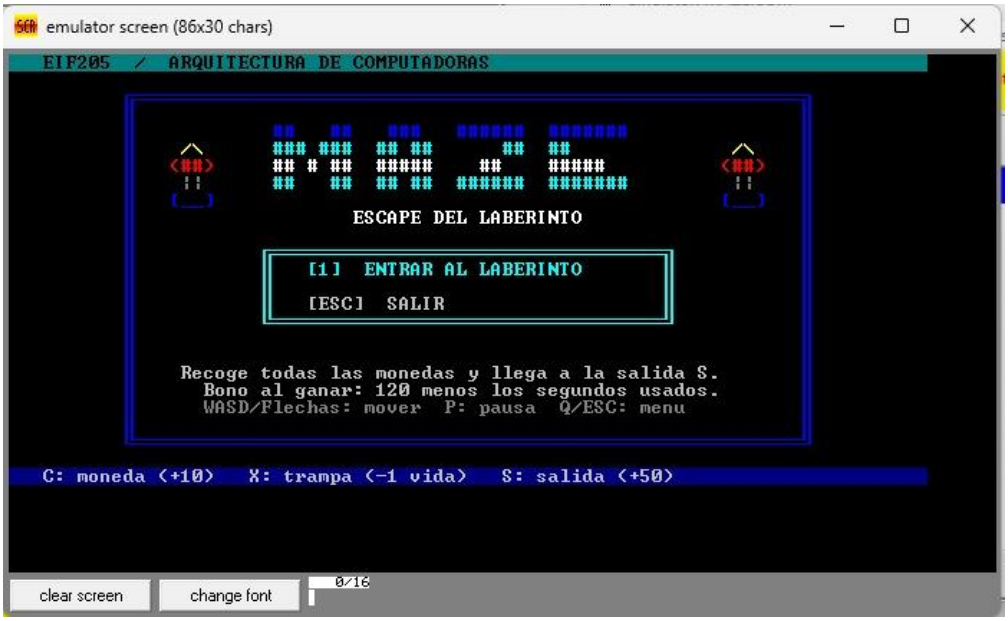


Figura E22. Menú en el emulador de EMU8086.

E23. Tablero en el emulador de EMU8086.

Con la ventana en 80 × 25 caracteres, el tablero y el marcador se ven completos, incluida la línea del tiempo.



Figura E23. Tablero en el emulador de EMU8086.

E24. Inicio de partida con el cronómetro.

Al comenzar, el marcador muestra el tiempo en 0 junto a los demás valores iniciales.



Figura E24. Inicio de partida con el cronómetro.

E25. Reloj avanzando durante la partida.

Después de recoger tres monedas, el tiempo marca 15 segundos. El valor se actualiza aunque el jugador no pulse teclas.



Figura E25. Reloj avanzando durante la partida.

E26. Pausa con el tiempo detenido.

La pantalla de pausa muestra los puntos, el récord y el tiempo al momento de pausar (17 segundos).



Figura E26. Pausa con el tiempo detenido.

E27. Partida reanudada.

Al continuar, la posición y el marcador son los mismos. El tiempo marca 18: el segundo transcurrido después de reanudar, no el tiempo en pausa.



Figura E27. Partida reanudada.

E28. Victoria con bono de tiempo.

La partida se ganó en 29 segundos. Los 191 puntos corresponden a 100 por monedas y salida más 91 de bono, y el récord se actualiza al mismo valor.



Figura E28. Victoria con bono de tiempo.

5.6. Evidencias de la versión anterior

Estas capturas corresponden a la primera ronda de pruebas. El marcador todavía no incluía la línea del tiempo, pero las reglas que comprueban (paredes, monedas, trampas, pausa, victoria, derrota y reinicio) son las mismas en la versión final.

E01. Compilación de Maze8086 en EMU8086.

El mensaje del ensamblador confirma que el programa se ensambló en cuatro pasadas y produjo un COM de 3376 bytes.

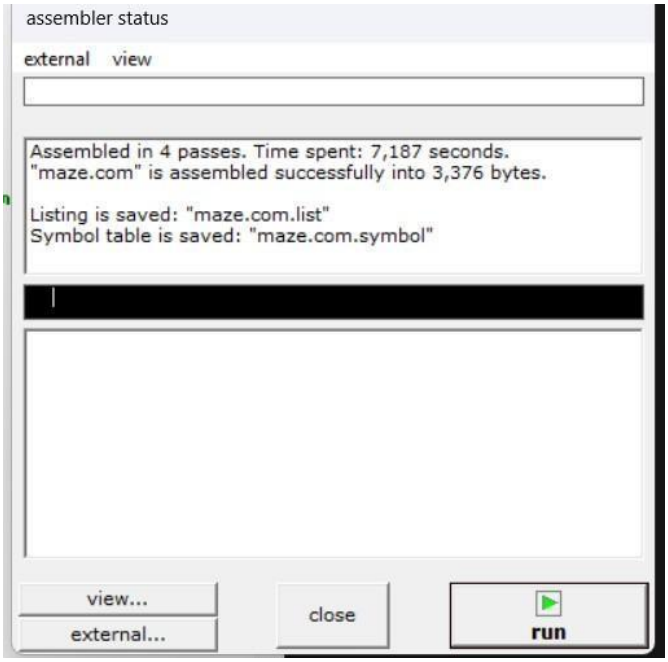


Figura E01. Compilación de Maze8086 en EMU8086.

E02. Menú principal.

Es la pantalla de entrada: la tecla 1 inicia una partida y ESC cierra el programa.



Figura E02. Menú principal.

E03. Tablero y marcador al comenzar una partida.

El marcador parte de 3 vidas, 5 monedas, 0 puntos y 0 movimientos. Estos valores sirven de referencia para las pruebas siguientes.



Figura E03. Tablero y marcador al comenzar una partida.

E04. Movimiento bloqueado por una pared.

La rutina de movimiento revisa la celda destino antes de cambiar las coordenadas. Al comparar con E03, la posición y el contador no cambian.



Figura E04. Movimiento bloqueado por una pared.

E05. Puntaje al recoger una moneda.

Al entrar en una celda con C se suman 10 puntos y las monedas pendientes bajan a 4.



Figura E05. Puntaje al recoger una moneda.

E06. Moneda recogida una sola vez.

La celda de la moneda queda vacía en el mapa de trabajo. Al regresar, el contador de movimientos aumenta pero el puntaje se mantiene en 10.



Figura E06. Moneda recogida una sola vez.

E07. Pantalla de pausa.

La tecla P cambia el estado a 4. Mientras dura la pausa, las teclas de movimiento no se procesan.



Figura E07. Pantalla de pausa.

E08. Continuación después de la pausa.

Al pulsar Enter el juego vuelve al estado 1 con la misma posición, los mismos puntos y los mismos movimientos que antes de la pausa.



Figura E08. Continuación después de la pausa.

E09. Nueva partida con el récord conservado.

Después de volver al menú e iniciar otra partida, el puntaje vuelve a 0 y el récord sigue en 10.



Figura E09. Nueva partida con el récord conservado.

E10. Pérdida de una vida por trampa.

Al entrar en la primera trampa quedan 2 vidas, el jugador regresa al inicio y la X desaparece del tablero.



Figura E10. Pérdida de una vida por trampa.

E11. Paso por una trampa ya activada.

La celda de la trampa quedó vacía, así que al pasar de nuevo por ella el jugador conserva 2 vidas.



Figura E11. Paso por una trampa ya activada.

E12. Intento de salida con monedas pendientes.

Con 2 monedas pendientes, el intento de entrar en S no mueve al jugador ni suma la bonificación. Las dos capturas muestran el marcador antes y después del intento.

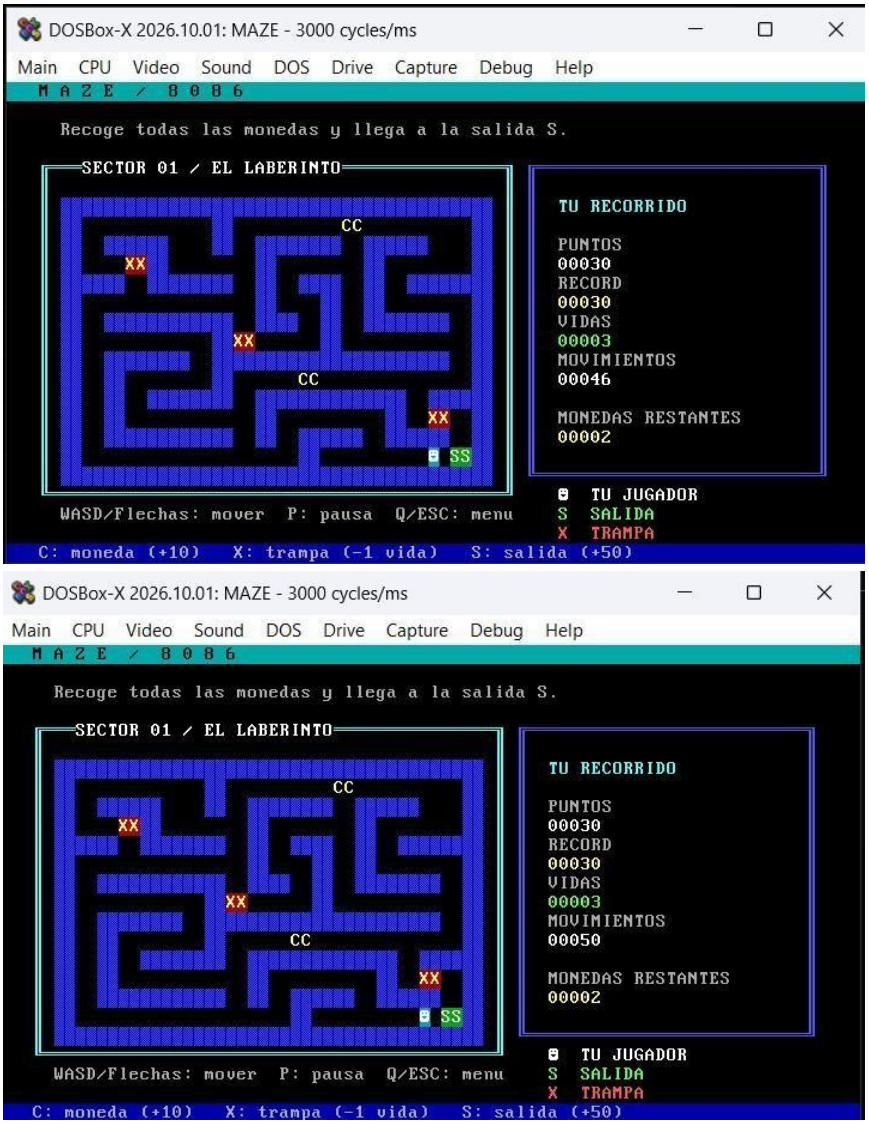


Figura E12. Intento de salida con monedas pendientes.

E13. Pantalla de victoria.

Con las cinco monedas recogidas, la salida suma 50 puntos y el juego muestra la victoria con 100 puntos y récord 100.



Figura E13. Pantalla de victoria.

E14. Partida restaurada después de la victoria.

La tecla R copia de nuevo el mapa original: vuelven las monedas y las trampas, y los contadores se reinician. El récord se mantiene en 100.



Figura E14. Partida restaurada después de la victoria.

E15. Pantalla de derrota.

Al perder la tercera vida el estado cambia a 3 y aparece la pantalla de Game Over con el récord de la sesión.



Figura E15. Pantalla de derrota.

E16. Nueva partida después de la derrota.

La tecla R reinicia la partida sin cerrar el emulador.



Figura E16. Nueva partida después de la derrota.

E17. Límites del laberinto.

El mapa tiene 20 columnas y 15 filas. Antes de leer una celda, la rutina de movimiento compara las coordenadas con esos límites, y el borde exterior del mapa es pared. El jugador permanece dentro del laberinto.



Figura E17. Límites del laberinto.

6. Juego de referencia y aportes del equipo

El equipo tomó como referencia el juego Escape Plan del repositorio MICROPROCESOR-EMU8086-ASSEMBLY-LANGUAGE. Ese juego usa un tablero de 20 × 20 y se controla con las flechas. Maze8086 usa un tablero propio de 20 × 15 y agrega controles WASD, monedas, trampas que se consumen, vidas, un marcador durante la partida, pausa, pantallas de victoria y derrota, récord, reinicio, cronómetro con bono de tiempo y efectos de sonido. La validación de movimiento usa un único cálculo de índice para las cuatro direcciones.

E18. Maze8086 (arriba) y el juego de referencia (abajo).

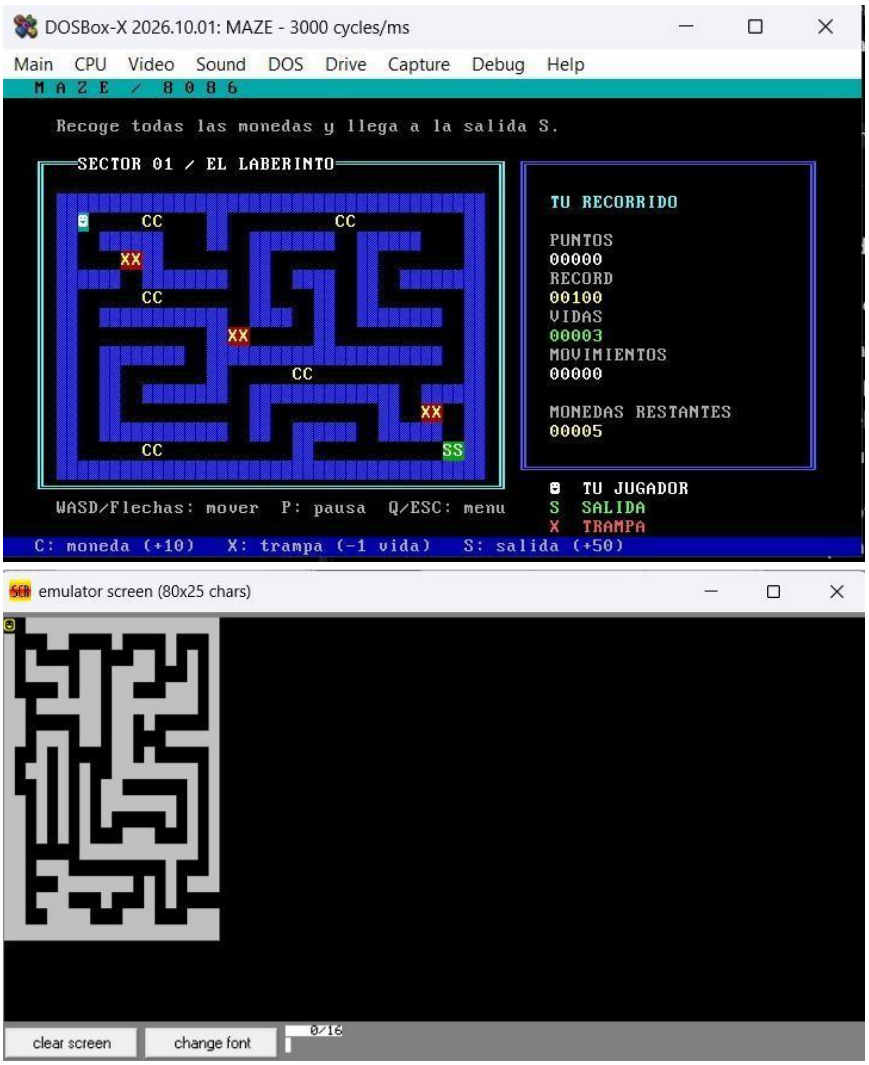


Figura E18. Maze8086 (arriba) y el juego de referencia (abajo).

7. Control de versiones

El proyecto se desarrolló en el repositorio https://github.com/belenS-tech/Maze8086-EIF205. Cada parte se trabajó en su propia rama y se unió a main cuando estuvo lista. Las ramas movimiento, juego e interfaz contienen los módulos originales; la rama integracion los conectó en un solo programa, y la rama cronometro-sonido agregó el cronómetro, el bono de tiempo, el sonido y la corrección de la alineación.

Commit	Autor y fecha	Contenido
fb3f2a1	Belén Sandoval, 20 oct	Estructura inicial del proyecto
de2a902	Juan José Fernández, 23 oct	Movimiento WASD y colisiones
462dd2a, PR #1	Isaac Altamirano, 28 oct	Vidas, puntuación y reinicio
8aee279	Belén Sandoval, 29 oct	Interfaz del laberinto
e466843 a 5224905	Belén Sandoval, 29 oct a 2 nov	Integración de los módulos, mejoras de la interfaz y documentación
7650710	Belén Sandoval, 8 nov	Informe técnico

Commit	Autor y fecha	Contenido
c116f23, PR #2	Isaac Altamirano, 8 nov	Cronómetro, bono de tiempo, módulo de sonido y alineación de datos
Actualiza MAZE.COM	Isaac Altamirano, 8 nov	Ejecutable compilado de la versión final

La figura E19 muestra el historial completo del repositorio con el autor de cada commit. Las ramas laterales corresponden a los pull requests: el PR #1 unió la rama juego y el PR #2 la rama cronometro-sonido; en ambos casos el merge lo hizo Isaac Altamirano.

E19. Historial de commits del repositorio por autor.

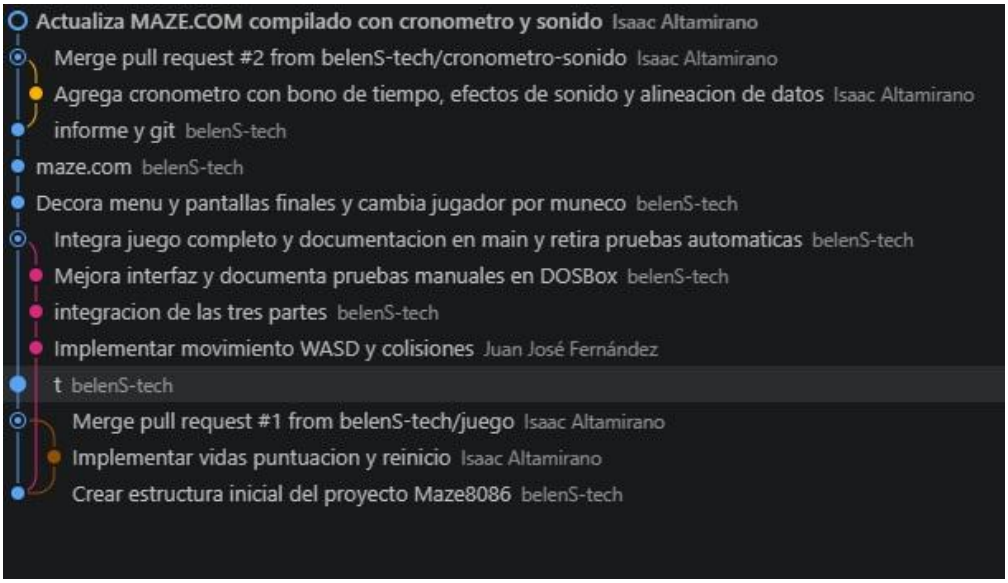


Figura E19. Historial de commits del repositorio por autor.

8. Conclusiones

El juego cumple las condiciones planteadas: tiene menú, partida, pausa, victoria y derrota como estados explícitos, y permite reiniciar sin cerrar el emulador. Las pruebas de la sección 5 muestran que las paredes y los bordes bloquean el paso, que las monedas y las trampas actúan una sola vez y que la salida exige recoger todas las monedas.

Separar el mapa en memoria de su dibujo en pantalla fue la decisión que más simplificó el trabajo. Las colisiones se resuelven consultando un byte del arreglo y la interfaz puede cambiar colores y caracteres sin afectar las reglas. Conservar una copia del mapa original permitió reiniciar la partida con una sola instrucción REP MOVSB.

La integración de los módulos mostró la importancia de acordar desde el inicio los nombres de las variables, los símbolos del mapa y qué registros preserva cada rutina. La mayoría de los errores de la sección 5.4 surgieron de esas diferencias y no de la lógica de cada módulo por separado.

El cronómetro obligó a cambiar la forma de leer el teclado. Esperar la tecla con INT 16h función 00h es sencillo, pero detiene el programa; para que el reloj avance hubo que consultar el teclado sin bloquear y leer el contador del BIOS en cada vuelta del ciclo. Medir el tiempo con ticks del

BIOS, en lugar de ciclos de retraso, hace que el reloj y la duración de los sonidos sean iguales en cualquier velocidad de emulación.

Revisar la tabla de símbolos permitió encontrar variables de 16 bits en direcciones impares, un problema que no producía errores visibles pero hacía más lento cada acceso en un 8086 real. También se comprobó que el emulador de EMU8086 muestra la memoria de video según el tamaño de su ventana, por lo que las pruebas en ese entorno deben hacerse en 80 × 25.

El juego tiene limitaciones conocidas: un solo nivel, un puntaje máximo de 220 por partida y un récord que se pierde al cerrar el programa porque no se guarda en disco.

Referencias

Equipo Maze8086. (2026). Maze8086-EIF205 [Repositorio de código]. GitHub. https://github.com/belenS-tech/Maze8086-EIF205

Ramaadarsh. (s. f.). MICROPROCESOR-EMU8086-ASSEMBLY-LANGUAGE [Repositorio de código]. GitHub. https://github.com/ramaadarsh/MICROPROCESOR-EMU8086-ASSEMBLY

Universidad Nacional. (2026). Proyecto 2: Proyecto de implementación de un juego en lenguaje ensamblador. Arquitectura de Computadoras (EIF205). Material del curso.